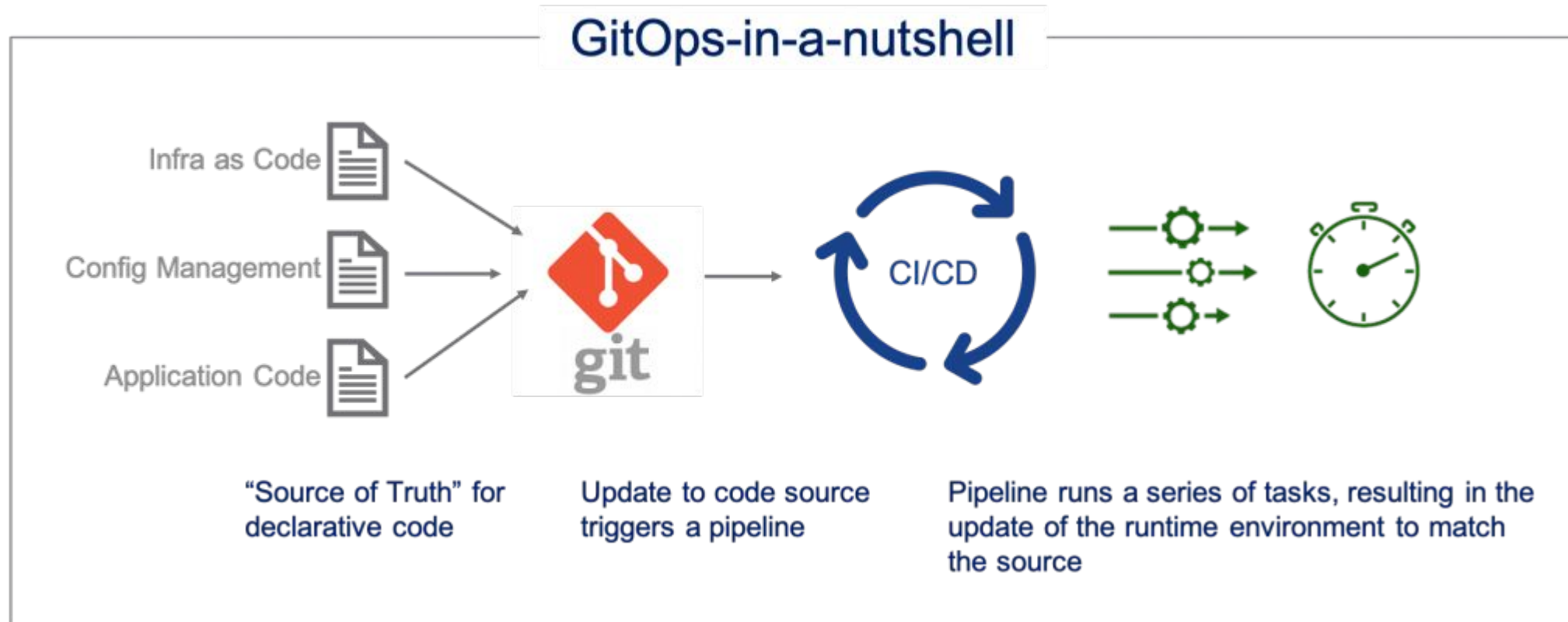
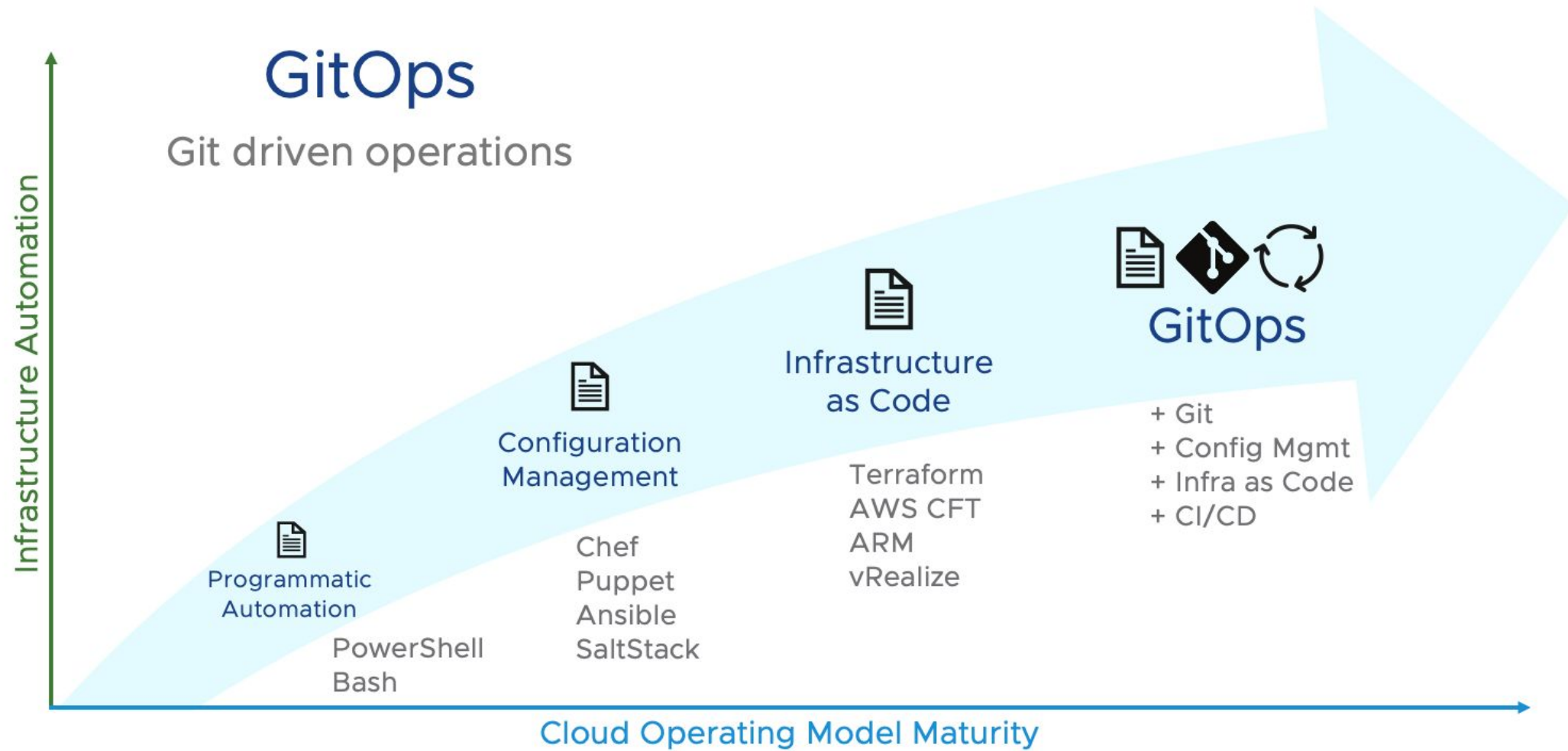




GitOps Overview

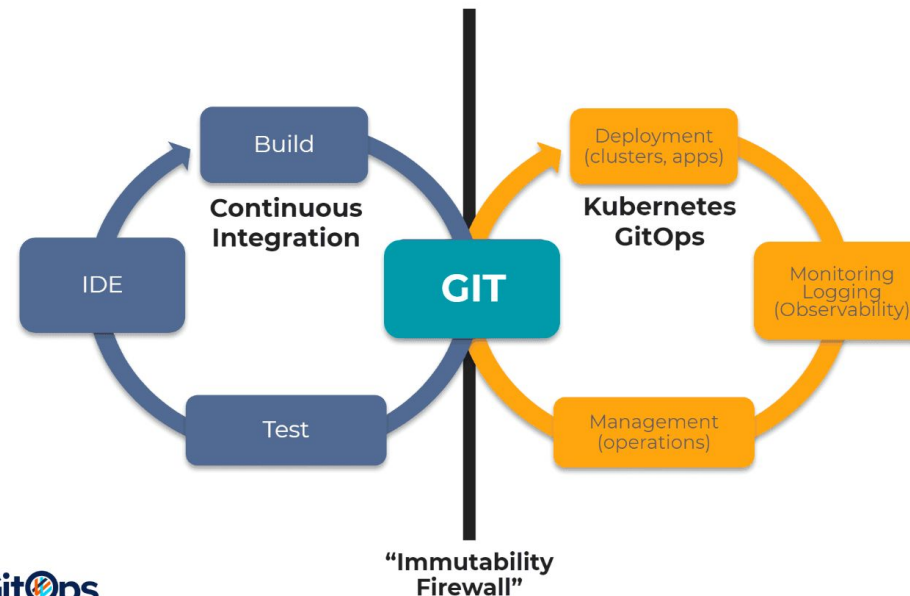


GitOps Overview



GitOps Overview

- GitOps is a software development methodology that uses Git as the single **source of truth** for all changes to a software system, including infrastructure, configuration, and application code. It enables developers to manage their infrastructure as code and automate the deployment process, improving efficiency, reliability, and scalability.
- One of the key benefits of GitOps is that it enables developers to manage their infrastructure as code, which means that infrastructure is treated as code and versioned, just like application code. This makes it easier to manage, test, and deploy changes to infrastructure, and reduces the risk of errors and downtime.



GitOps Tools



What is Argo CD?

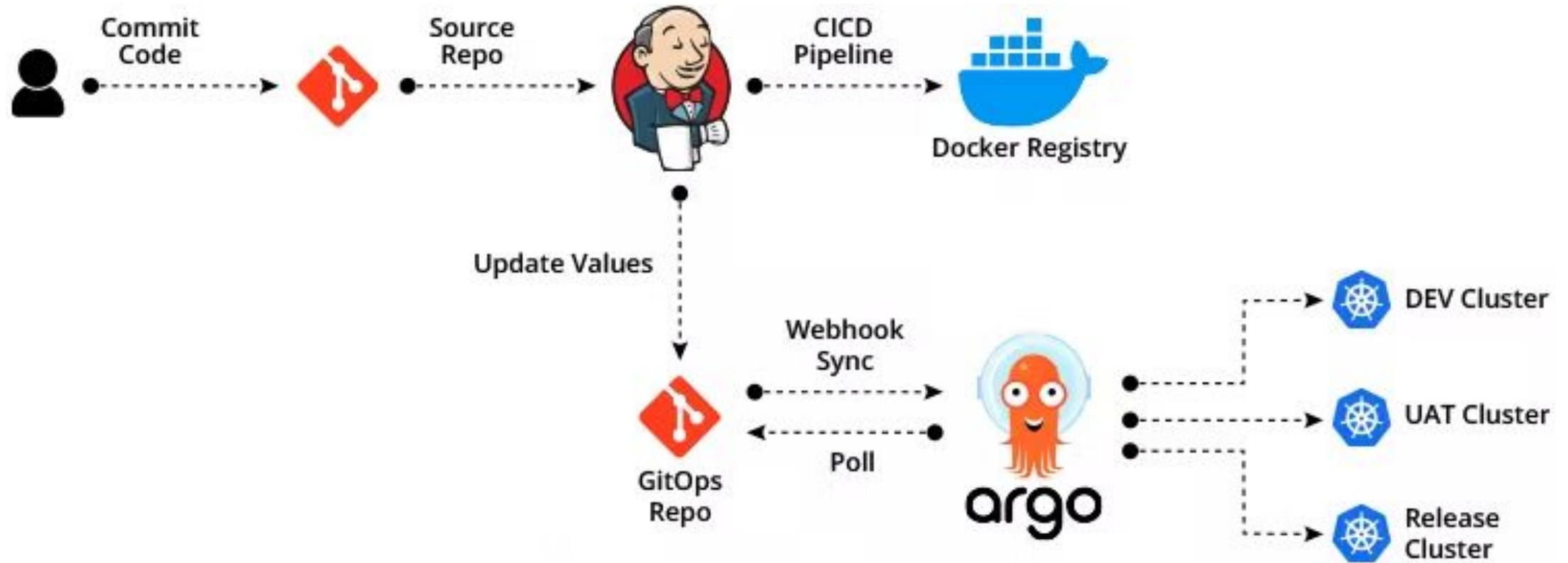


Image 1: CI/CD Flow with Argo CD

What is Argo CD?

- Argo CD is a declarative, GitOps continuous delivery tool for Kubernetes.
- Argo CD is used for Application definitions, configurations, and environments should be declarative and version controlled. Application deployment and lifecycle management should be automated, auditable, and easy to understand.
- Argo CD automates the deployment of the desired application states in the specified target environments. Application deployments can track updates to branches, tags, or pinned to a specific version of manifests at a Git commit. See [tracking strategies](#) for additional details about the different tracking strategies available.



Argo CD Features

- **Features**

- Automated deployment of applications to specified target environments
- Support for multiple config management/templating tools (Kustomize, Helm, Jsonnet, plain-YAML)
- Ability to manage and deploy to multiple clusters
- SSO Integration (OIDC, OAuth2, LDAP, SAML 2.0, GitHub, GitLab, Microsoft, LinkedIn)
- Multi-tenancy and RBAC policies for authorization
- Rollback/Roll-anywhere to any application configuration committed in Git repository
- Health status analysis of application resources
- Automated configuration drift detection and visualization
- Automated or manual syncing of applications to its desired state
- Web UI which provides real-time view of application activity
- CLI for automation and CI integration
- Webhook integration (GitHub, BitBucket, GitLab)
- Access tokens for automation
- PreSync, Sync, PostSync hooks to support complex application rollouts (e.g.blue/green & canary upgrades)
- Audit trails for application events and API calls
- Prometheus metrics
- Parameter overrides for overriding helm parameters in Git



Argo CD Core Concepts

- Let's assume you're familiar with core Git, Docker, Kubernetes, Continuous Delivery, and GitOps concepts. Below are some of the concepts that are specific to Argo CD.
 - **Application** A group of Kubernetes resources as defined by a manifest. This is a Custom Resource Definition (CRD).
 - **Application source type** Which Tool is used to build the application.
 - **Target state** The desired state of an application, as represented by files in a Git repository.
 - **Live state** The live state of that application. What pods etc are deployed.
 - **Sync status** Whether or not the live state matches the target state. Is the deployed application the same as Git says it should be?
 - **Sync** The process of making an application move to its target state. E.g. by applying changes to a Kubernetes cluster.
 - **Sync operation status** Whether or not a sync succeeded.
 - **Refresh** Compare the latest code in Git with the live state. Figure out what is different.
 - **Health** The health of the application, is it running correctly? Can it serve requests?
 - **Tool** A tool to create manifests from a directory of files. E.g. Kustomize. See Application Source Type.
 - **Configuration management tool** See Tool.
 - **Configuration management plugin** A custom tool.



Argo CD Ingress with TLS

- Change Argo CD Service to **ClusterIP** if you have changed it.

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: argocd-clusterissuer
  namespace: argocd
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: leangsengk90@gmail.com
    privateKeySecretRef:
      name: argocd-letsencrypt-key
    solvers:
      - http01:
          ingress:
            class: nginx
```

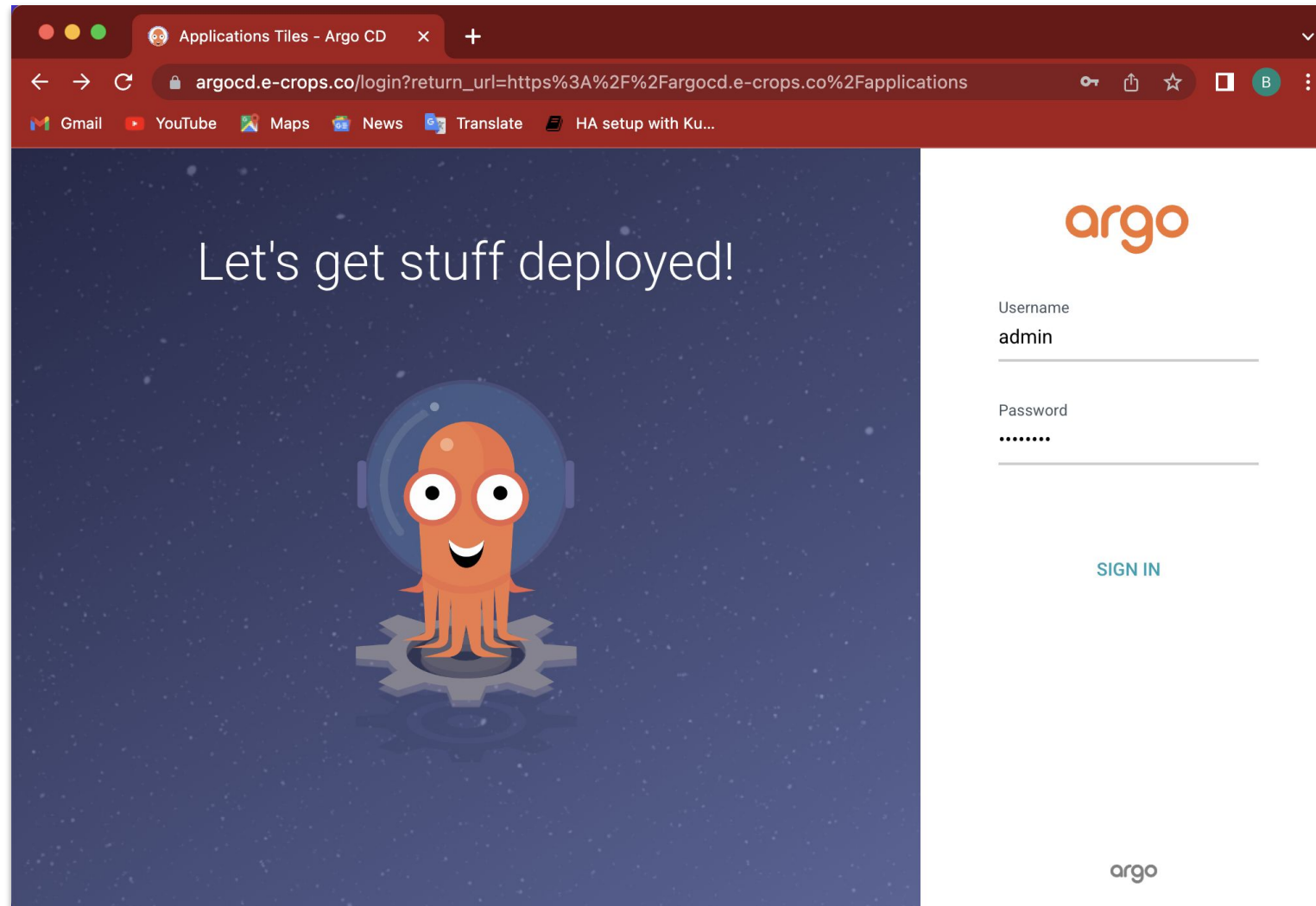


Argo CD Ingress with TLS

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: argocd-server-ingress
  namespace: argocd
  annotations:
    cert-manager.io/cluster-issuer: argocd-clusterissuer
    kubernetes.io/ingress.class: nginx
    kubernetes.io/tls-acme: "true"
    nginx.ingress.kubernetes.io/ssl-passthrough: "true"
    # If you encounter a redirect loop or are getting a 307 response code
    # then you need to force the nginx ingress to connect to the backend using HTTPS.
    nginx.ingress.kubernetes.io/backend-protocol: "HTTPS"
spec:
  rules:
    - host: argocd.e-crops.co
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: argocd-server
                port:
                  name: https
  tls:
    - hosts:
        - argocd.e-crops.co
      secretName: argocd-secret # do not change, this is provided by Argo CD
```

Argo CD Ingress with TLS

- `$ kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d`



Argo CD with Jenkins

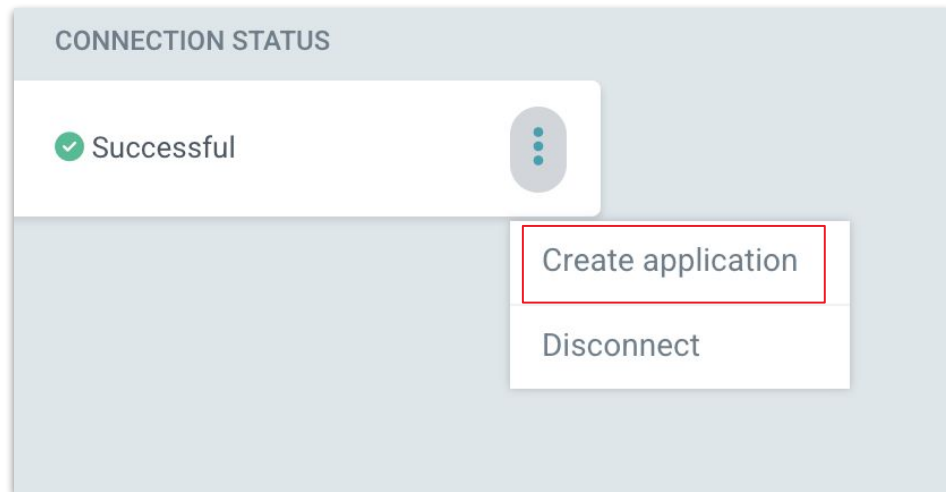
```
# First Repo: Your Project Code Repo
pipeline{
  agent { label 'master' }
  stages{
    stage("Build Docker Image"){
      steps{
        script{
          docker.withRegistry('', 'dockerhub-auth') {
            def customImage = docker.build("xeng/react-router:${BUILD_NUMBER}")
            /* Push the container to the custom Registry */
            customImage.push()
          }
        }
        sh 'docker logout'
        //Trigger to build other job => Get value: "${params.VERSION}"
        build job: 'frontend-argocd', parameters: [
          string(name: 'VERSION', value: "${BUILD_NUMBER}")
        ]
      }
    }
  }
}
```

Argo CD with Jenkins

```
# Second Repo: Your Kubernetes Yaml files Repo
pipeline {
  agent { label 'master' }
  stages {
    stage('Modify Deployment Configuration') {
      steps {
        sh """
          sed -i 's+xeng/react-router:.*+xeng/react-router:${params.VERSION}+g' frontend/deployment.yml
          git config user.name 'leangsengk90'
          git config user.email 'leangsengk90@gmail.com'
          git add .
          git commit -m 'change file ${params.VERSION}'
          git push
          https://leangsengk90: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@github.com/leangsengk90/argocd.git HEAD:main
        """
      }
    }
  }
}
```

Argo CD Application

- Setting → Repositories → Connect Repo
- Click: **CONNECT**
- Click: **Create Application**



CONNECT SAVE AS CREDENTIALS TEMPLATE CANCEL

Choose your connection method:

VIA HTTPS ▼

CONNECT REPO USING HTTPS

Type

git

Project

default

Repository URL

https://github.com/leangsengk90/argocd.git

Username (optional)

admin

Password (optional)

.....|



Argo CD Application

- All done click: **CREATE** and Build Your Project Job

CREATECANCEL

GENERAL

Application Name
frontend-cd

Project Name
default

SYNC POLICY

Automatic

☒ PRUNE RESOURCES

☒ SELF HEAL

☐ SET DELETION FINALIZER

SYNC OPTIONS

☐ SKIP SCHEMA VALIDATION

☐ PRUNE LAST

☐ RESPECT IGNORE DIFFERENCES

☐ AUTO-CREATE NAMESPACE

☐ APPLY OUT OF SYNC ONLY

☐ SERVER-SIDE APPLY

PRUNE PROPAGATION POLICY: foreground

☐ REPLACE

☐ RETRY

SOURCE

Repository URL
https://github.com/leangsengk90/argocd.git

Revision
HEAD

Path
frontend

DESTINATION

Cluster URL
https://kubernetes.default.svc

Namespace
default



Argo CD Application

```
# Create Argo CD with Yaml file for Auto Sync
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: frontend-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/leangsengk90/argocd.git
    targetRevision: HEAD
    path: ./frontend
  destination:
    namespace: default
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - Validate=false
```

Argo CD Application

```
# Create Argo CD with Yaml file for Manual Sync
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: frontend-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/leangsengk90/argocd.git
    targetRevision: HEAD
    path: ./frontend
  destination:
    namespace: default
    server: https://kubernetes.default.svc
  syncPolicy:
    syncOptions:
      - Validate=false
```

Argo CD Application

- Click: **Applications** and **Sync**

Argo CD

v2.6.3+e05298b

Applications

Settings

User Info

Documentation

FILTERS

NAME

KINDS

SYNC STATUS

HEALTH STATUS

Applications / frontend-cd

APP DETAILS APP DIFF SYNC SYNC STATUS HISTORY AND ROLLBACK DELETE REFRESH

APP HEALTH Healthy

CURRENT SYNC STATUS Synced To HEAD (a36009c)

LAST SYNC RESULT Sync OK

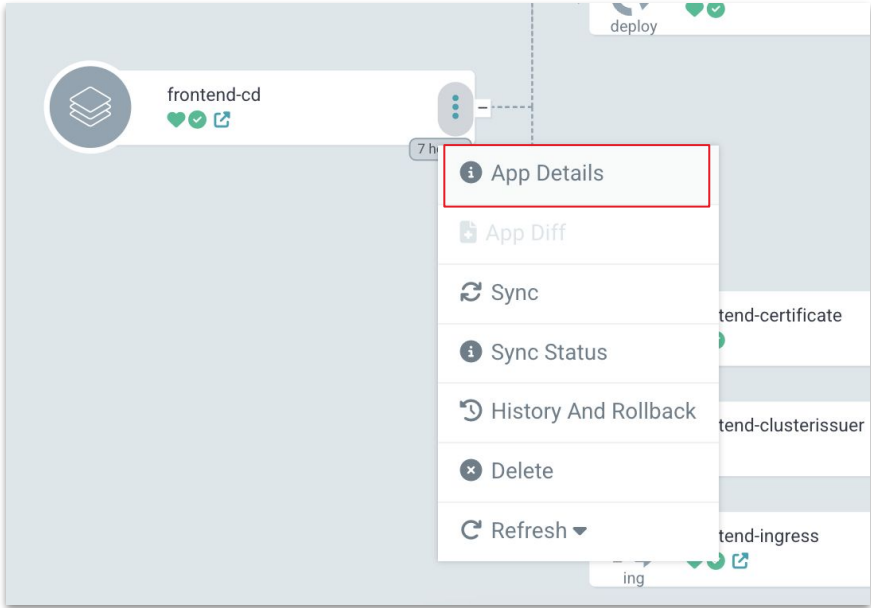
frontend-cd

default-configmap frontend-svc frontend-deploy frontend-certificate frontend-clusterissuer frontend-ingress frontend-svc frontend-svc-hmkdj frontend-deploy-55cdf8d7c6 frontend-deploy-74c668ccd5 frontend-deploy-76bc7f9575 frontend-deploy-887b6b666 frontend-deploy-887b6b666-4 frontend-certificate-f8zsl frontend-certificate-f8zsl-880



Argo CD Application

- Click **APP DETAILS**: disable or enable Sync Policy



FRONTEND-CD	
PROJECT	default
ANNOTATIONS	
CLUSTER	in-cluster (https://kubernetes.default.svc)
NAMESPACE	default
CREATED AT	03/25/2023 23:35:26 (7 hours ago)
REPO URL	https://github.com/leangsengk90/argocd.git
TARGET REVISION	HEAD
PATH	frontend
SYNC OPTIONS	
RETRY OPTIONS	Retry disabled
STATUS	✔ Synced To HEAD (a36009c)
HEALTH	♥ Healthy
LINKS	
URLs	https://frontend.e-crops.co/
IMAGES	xeng/react-router:68
SYNC POLICY	
AUTOMATED	DISABLE AUTO-SYNC
PRUNE RESOURCES	DISABLE
SELF HEAL	DISABLE



Argo CD with Git Webhook

- Argo CD polls Git repositories **every three minutes** to detect changes to the manifests. To eliminate this delay from polling, the API server can be configured to receive webhook events. Argo CD supports Git webhook notifications from GitHub, GitLab, Bitbucket, Bitbucket Server and Gogs. The following explains how to configure a Git webhook for GitHub, but the same process should be applicable to other providers.
- In your Git provider, navigate to the settings page where webhooks can be configured. The payload URL configured in the Git provider should use the **/api/webhook** endpoint of your Argo CD instance (e.g. **https://argocd.example.com/api/webhook**). If you wish to use a shared secret, input an arbitrary value in the secret. This value will be used when configuring the webhook in the next step.

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

Content type

Secret

☐ By default, we verify SSL certificates when delivering payloads. [Disable SSL verification](#)

Which events would you like to trigger this webhook?

☒ Just the push event.

☐ Send me everything.

☐ Let me select individual events.

☒ **Active**
We will deliver event details when this hook is triggered.

[Add webhook](#)



Argo CD with Git Webhook

```
$ kubectl edit secret argocd-secret -n argocd
apiVersion: v1
kind: Secret
metadata:
  name: argocd-secret
  namespace: argocd
type: Opaque
data:
---
stringData:
  # github webhook secret
  webhook.github.secret: "github123"
  # gitlab webhook secret
  webhook.gitlab.secret: gitlab123
  # bitbucket webhook secret
  webhook.bitbucket.uuid: your-bitbucket-uuid
  # bitbucket server webhook secret
  webhook.bitbucketserver.secret: bitbucket123
  # gogs server webhook secret
  webhook.gogs.secret: gogs123
```

Provider	K8s Secret Key
GitHub	webhook.github.secret
GitLab	webhook.gitlab.secret
BitBucket	webhook.bitbucket.uuid
BitBucketServer	webhook.bitbucketserver.secret
Gogs	webhook.gogs.secret

Sync Window

- In ArgoCD, "window sync" refers to a feature that allows users to control when the application deployment should happen. By default, when an application's configuration is updated in the source repository, ArgoCD will automatically detect the changes and initiate a deployment to update the application. However, in some cases, users may want more control over when the deployment occurs.
- With window sync, users can set a "sync window" during which time automatic deployments will be allowed. For example, a user may set a sync window of 9:00 AM to 5:00 PM, during which time automatic deployments are allowed. Any changes made to the application's configuration outside of the sync window will not trigger an automatic deployment.
- This feature can be useful in environments where changes to the application's configuration can have a significant impact on users or the overall system, and where it is important to have more control over when those changes are deployed. It can help prevent unexpected deployments during periods of high traffic or user activity, and allow for more deliberate and controlled management of the deployment process.

Sync Window

CREATE

CANCEL

Kind

allow

Schedule

MINUTE ⓘ

Every Minute

0

1

2

3

4

5

6

7

HOUR ⓘ

15

16

17

18

19

20

21

22

23

DAY OF THE MONTH ⓘ

Every Day

1

2

3

4

5

6

7

8

MONTH ⓘ

Every Month

Jan

Feb

Mar

Apr

May

Jun

Jul

Aug

DAY OF THE WEEK ⓘ

Sunday-Saturday

Sun

Mon

Tue

Wed

Thu

Fri

Sat

Time Zone

Asia/Bangkok

Duration (e.g. "30m" or "1h")

1h

☒ Enable manual sync

APPLICATIONS

Manage applications assigned to this window ("*" for any)

*

ADD APPLICATION

+ ADD ROLE

+ ADD SYNC WINDOW

DELETE

SUMMARY

ROLES

WINDOWS

EVENTS

STATUS ⓘ	WINDOW ⓘ	APPLICATIONS ⓘ	NAMESPACES ⓘ	CLUSTERS ⓘ	MANUALSYNC ⓘ
Inactive	allow:0 20 ***:1h:Asia...	*	-	-	Disabled

- Settings → Projects → Click: Your Project
- Click: “+ **ADD SYNC WINDOWS**” button
- **Kind** options:
 - allow
 - **Red: sync denied**
 - ✓ Enable manual sync
 - **Orange: manual allowed**
 - deny
 - **Green: sync allowed**

A large, faint watermark of the ISTAD logo is centered in the background. It is a circular emblem with a blue outer ring containing the text 'INSTITUTE OF SCIENCE AND TECHNOLOGY ADVANCED DEVELOPMENT' and Khmer text. Inside the ring is a blue circle with a white atomic symbol. The word 'ISTAD' is written in red across the bottom of the inner circle. Binary code '01010100 01000001 01000000' is visible along the top inner edge of the ring.

Thank you

Begin with the theory, Start with the practical